



400-650-7088

# ***Tong*Search** V3.0

## 性能优化手册

版本：3061A01

2025年09月

# 声明

## 版权

Copyright ©2025北京东方通科技股份有限公司或其关联公司（以下简称“东方通”）。保留所有权利。

## 版权声明

- 本文档的所有权和知识产权归属于东方通所有。
- 除非另有明确约定，东方通不授予任何明示或暗示的许可或权利，使用者不得以任何方式将本文档中的任何内容用于商业目的。
- 东方通对本文档享有版权。本文档的所有内容，包括但不限于文字、图像、图表、图标、示意图、屏幕截图等，均受版权法和国际版权条约的保护。
- 未经东方通明确授权，任何人不得对本文档以任何形式复制、修改、传播、分发、展示或进行衍生创作。

## 商标声明

- **东方通**、**TongTech** 标识以及其他相关东方通图形、徽标、服务名称和商标（以下统称为“东方通商标”）是东方通或其关联公司的注册商标或商标。
- 未经东方通明确授权，任何人不得以任何方式使用、复制或展示东方通商标。此外，未经东方通事先书面同意，不得将东方通商标与其他商标、标识或徽标进行混淆、链接或结合使用。
- 除非另有明确约定，本商标声明不授予任何明示或暗示的许可或权利，对东方通商标的使用须获得东方通的明确授权。
- 本文档提及的所有其他商标、标识、徽标或产品名称均为其各自所有者的财产，并可能受其各自的商标法保护。

## 免责声明

请在使用东方通产品之前仔细阅读本免责声明，并根据自身情况判断是否继续使用。如有任何问题，请联系我们的客户支持团队。

- 本产品的使用是基于用户自己的判断和风险评估。本文档仅作为使用指导，不对使用本产品所产生的结果做任何明示或暗示的担保。
- 东方通不对用户未按照本文档中的指导正确使用东方通产品而导致的任何损失或损害承担责任；
- 本文档可能会包含第三方提供的内容、链接或资源，这些内容由第三方自行负责。东方通对于这些内容的准确性、完整性、合法性或可靠性不承担责任。用户在使用这些内容时应自行判断和承担风险。
- 由于产品版本升级或其他原因，本文档内容会不定期更新。东方通保留在不事先通知用户的情况下随时对文档进行修改、更新或中止的权利。

如需获取有关东方通产品的许可或使用权，请联系东方通或授权代理商。任何违反本声明的行为将受到适用法律的追究。

# 版本变更说明

本手册的更新是累积的。因此，最新的手册版本包含对以前版本所做的所有更改。

| 手册版本    | 适用产品    | 更新内容                                      |
|---------|---------|-------------------------------------------|
| 3061A01 | 3.0.6.1 | 修改版权信息                                    |
| 3060A01 | 3.0.6.0 | 新增如下章节：<br><a href="#">慢查询日志转存</a>        |
| 3050A01 | 3.0.5.0 | 无                                         |
| 3040A01 | 3.0.4.0 | 无                                         |
| 3030A01 | 3.0.3.0 | 新增如下章节：<br><a href="#">通过开启慢查询配置定位慢查询</a> |
| 3020A01 | 3.0.2.0 | 无                                         |
| 3010A01 | 3.0.1.0 | 首次发版。                                     |

# 前言

本文档是 TongSearch 产品的用户手册之一，介绍了在使用 TongSearch 过程中的写优化、读优化、服务器配置优化、数据结构优化以及集群架构优化建议。

## 阅读前注意事项

通过阅读本文档，您确认并同意自行承担因未具备必要专业背景 and 知识而导致的任何风险或后果。在使用本文档中提供的信息和指南时，请始终谨慎，并在必要时寻求专业人士建议和指导。

- 适用对象

本文档主要适用于使用本产品的系统管理员阅读，部分内容同样适用于基于本产品进行应用开发或应用部署的人员阅读。

- 专业背景

本文内容可能涉及到操作系统、服务器硬件、网络等相关领域的知识。请确保您具备相关背景 and 知识，以便更好地理解 and 应用本手册的内容。

- 技能要求

为了能够充分理解 and 应用本文档的内容，建议您具备如下技能：

- 基本系统管理任务
- 掌握 Linux 系统基本操作
- 在终端窗口发布命令
- 使用 Web 浏览器
- 安装软件

- 术语和概念

本文档可能使用一些专业术语和概念。请确保您熟悉这些术语和概念，或者有能力查阅相关资料以便进一步理解。

- 实践经验

为了最大程度地受益于本文档，建议您具备一定实践经验。这将帮助您更好地应用文档中的操作指南 and 建议。

请注意，本文档不适用于没有相关专业背景 and 知识的用户。如果您对本文档的内容 or 所需背景有任何疑问，请在使用之前咨询相关专业人士 or 寻求额外的支持。

## 用户手册集

TongSearch V3.0 为您提供的用户使用手册集包含的文档有：

| 编号 | 手册集文档                    | 说明                                   |
|----|--------------------------|--------------------------------------|
| 1  | 000_TongSearch_V3.0产品白皮书 | 介绍了 TongSearch 的产品概述、产品功能、产品特色及产品小结。 |

| 编号 | 手册集文档                          | 说明                                                               |
|----|--------------------------------|------------------------------------------------------------------|
| 2  | 001_TongSearch_V3.0产品介绍手册      | 介绍了 TongSearch 概述、功能、场景、基本概念等。                                   |
| 3  | 002_TongSearch_V3.0快速使用手册      | 介绍了如何快速上手 TongSearch。                                            |
| 4  | 003_TongSearch_V3.0产品安装手册      | 提供 TongSearch 常见场景的部署方式及示例程序使用说明。                                |
| 5  | 004_TongSearch_V3.0集群管理手册      | 提供 TongSearch 集群搭建、灾备、升级说明。                                      |
| 6  | 005_TongSearch_V3.0DSL使用手册     | 提供 TongSearch 产品的 DSL 语句使用说明。                                    |
| 7  | 006_TongSearch_V3.0EQL使用手册     | 提供 TongSearch 产品的 EQL 语句使用说明。                                    |
| 8  | 007_TongSearch_V3.0SQL使用手册     | 提供 TongSearch 产品的 SQL 语句使用说明。                                    |
| 9  | 008_TongSearch_V3.0脚本使用手册      | 提供命令行工具的使用说明，所有命令行工具的功能说明、相关参数、实例及验证方法。                          |
| 10 | 009_TongSearch_V3.0聚合使用手册      | 介绍了如何使用聚合语句操作 TongSearch 中存储的数据。                                 |
| 11 | 010_TongSearch_V3.0REST API 手册 | 提供了通过 HTTP 公开的 REST API。                                         |
| 12 | 011_TongSearch_V3.0Java SDK 手册 | 提供基于 Java 语言的客户端使您能够轻松的访问 TongSearch 的 API。                      |
| 13 | 012_TongSearch_V3.0工具使用手册      | 介绍了 TongSearch 提供的各种脚本工具使用说明。                                    |
| 14 | 013_TongSearch_V3.0数据迁移手册      | 提供 Elasticsearch 中存储的数据向 TongSearch 迁移的方案。                       |
| 15 | 014_TongSearch_V3.0安全管理手册      | 提供 TongSearch 安全特性使您能够轻松保护集群。                                    |
| 16 | 015_TongSearch_V3.0性能优化手册      | 介绍了在使用 TongSearch 过程中的写优化、读优化、服务器配置优化、数据结构优化以及集群架构优化建议。          |
| 17 | 016_TongSearch_V3.0监控告警手册      | 提供了 Prometheus、Alertmanager 以及 Grafana 结合使用对 TongSearch 进行监控的说明。 |
| 18 | 017_TongSearch_V3.0常见问题手册      | 提供使用 TongSearch 过程中遇到的常见问题及解决方案。                                 |

| 编号 | 手册集文档                               | 说明                                                        |
|----|-------------------------------------|-----------------------------------------------------------|
| 19 | 018_TongSearch_V3.0控制台使用手册          | 提供管理和监视 TongSearch 集群的用户界面，旨在简化 TongSearch 的配置、维护和监控。     |
| 20 | 019_TongSearch_V3.0TongPipeline使用手册 | 提供 TongPipeline 详细的使用说明，帮助用户快速上手 TongPipeline 并深入了解其各项功能。 |
| 21 | 020_TongSearch_V3.0插件使用手册           | 提供 TongSearch 插件详细的使用说明。                                  |

技术支持

东方通产品将为您提供全方位的技术支持，您可以通过以下方式获得技术支持：

- 网址：[www.tongtech.com](http://www.tongtech.com)
- 电话：400-650-7088
- 邮箱：[support@tongtech.com](mailto:support@tongtech.com)

您在取得技术支持时，需提供如下信息：

- 您的姓名
- 您的公司信息
- 您的联系方式
- 操作系统及其版本
- 产品版本号
- 日志等错误的详细信息

# 目录

|                                      |    |
|--------------------------------------|----|
| 声明 .....                             | 1  |
| 版本变更说明 .....                         | 2  |
| 前言 .....                             | 3  |
| 1. 性能优化概述 .....                      | 8  |
| 2. 写优化 .....                         | 9  |
| 2.1. 批量提交 .....                      | 9  |
| 2.2. 优化存储设备 .....                    | 9  |
| 2.3. 减少 Refresh 的次数 .....            | 9  |
| 2.4. 加大 Flush 设置 .....               | 10 |
| 2.5. 减少副本的数量 .....                   | 10 |
| 2.6. 修改 index_buffer_size 的设置 .....  | 11 |
| 2.7. 注意 _id 字段的使用 .....              | 11 |
| 2.8. 注意 _all 字段及 _source 字段的使用 ..... | 11 |
| 2.9. 合理的配置使用 index 属性 .....          | 11 |
| 3. 读优化 .....                         | 12 |
| 3.1. 避免大结果集和深翻 .....                 | 12 |
| 3.2. 选择合适的路由 .....                   | 13 |
| 3.3. SearchType .....                | 13 |
| 3.4. Filter VS Query .....           | 14 |
| 3.5. 脚本（script）合理使用 .....            | 14 |
| 3.6. Cache的设置及使用 .....               | 14 |
| 3.7. 定期删除 .....                      | 15 |
| 3.8. 更多查询优化经验 .....                  | 15 |
| 4. 服务器配置 .....                       | 17 |
| 4.1. CPU 配置 .....                    | 17 |
| 4.2. 堆大小的设置 .....                    | 17 |
| 4.3. 服务器配置的选择 .....                  | 18 |
| 4.4. GC 设置 .....                     | 19 |
| 4.5. 磁盘选择 .....                      | 19 |
| 4.6. 角色隔离 .....                      | 20 |
| 4.7. 通过开启慢查询配置定位慢查询 .....            | 20 |
| 4.7.1. 慢查询日志字段说明 .....               | 21 |
| 4.7.2. 慢查询日志转存 .....                 | 21 |
| 4.7.2.1. 创建存储慢查询日志的索引 .....          | 21 |
| 4.7.2.2. 配置慢查询转发 .....               | 22 |
| 4.7.2.3. 存储的慢查询日志字段说明 .....          | 23 |

4.7.2.4. 存储的索引数据时产生的慢日志字段说明..... 24

**5. 数据结构优化..... 26**

5.1. 尽量减少不需要的字段..... 26

5.2. 嵌套对象与父/子对象..... 26

5.3. 选择静态映射、非必需时，禁止动态映射..... 27

5.4. document 模型设计..... 27

**6. 集群架构设计..... 28**

6.1. 主节点、数据节点和协调节点分离..... 28

6.2. 关闭 data 节点服务器中的 http 功能 ..... 29

6.3. 一台服务器上最好只部署一个 node ..... 30

6.4. 集群分片设置..... 30



# 1. 性能优化概述

TongSearch 的默认配置，是综合了数据可靠性、写入速度、搜索实时性等因素。实际使用时，我们需要根据公司要求，进行偏向性的优化。

## 2. 写优化

假设我们的应用场景要求是，每秒 300 万的写入速度，每条 500 字节左右。

针对这种对于搜索性能要求不高，但是对写入要求较高的场景，我们需要尽可能的选择恰当写优化策略。

综合来说，可以考虑以下几个方面来提升写索引的性能：

- 加大 Translog Flush，目的是降低 IOPS、Writeblock。
- 增加 Index Refresh 间隔，目的是减少 Segment Merge 的次数。
- 调整 Bulk 线程池和队列。
- 优化节点间的任务分布。
- 优化 Lucene 层的索引建立，目的是降低 CPU 及 IO。

### 2.1. 批量提交

TongSearch 提供了 Bulk API 支持批量操作，当我们有大量的写任务时，可以使用 Bulk 来进行批量写入。每次提交的数据量为多少时，能达到最优的性能，主要受到文件大小、网络情况、数据类型、集群状态等因素影响。

**通用的策略如下：** Bulk 默认设置批量提交的数据量不能超过 100M。数据条数一般是根据文档的大小和服务器的性能而定的，但是单次批处理的数据大小应从 5MB ~ 15MB 逐渐增加，当性能没有提升时，把这个数据量作为最大值。

### 2.2. 优化存储设备

TongSearch 是一种密集使用磁盘的应用，在段合并的时候会频繁操作磁盘，所以对磁盘要求较高，当磁盘速度提升之后，集群的整体性能会大幅度提高。

磁盘的选择，提供以下几点建议：

- 使用固态硬盘（Solid State Disk）替代机械硬盘。SSD 与机械磁盘相比，具有高效的读写速度和稳定性。
- 使用 RAID 0。RAID 0 条带化存储，可以提升磁盘读写效率。
- 在 TongSearch 的服务器上挂载多块硬盘。使用多块硬盘同时进行读写操作提升效率，在配置文件 TongSearch 中设置多个存储路径，如下所示：

```
path.data:/path/to/data1,/path/to/data2
```



避免使用 NFS（Network File System）等远程存储设备，网络的延迟对性能的影响是很大的。

### 2.3. 减少 Refresh 的次数

Lucene 在新增数据时，采用了延迟写入的策略，默认情况下索引的 refresh\_interval 为 1 秒。

Lucene 将待写入的数据先写到内存中，超过 1 秒（默认）时就会触发一次 Refresh，然后 Refresh 会把内

存中的的数据刷新到操作系统的文件缓存系统中。

如果我们对搜索的实效性要求不高，可以将 Refresh 周期延长，例如 30 秒。

这样还可以有效地减少段刷新次数，但这同时意味着需要消耗更多的Heap内存。

如下所示。

```
PUT /{index}/_settings
{
  "refresh_interval": "30s"
}
```

## 2.4. 加大 Flush 设置

Flush 的主要目的是把文件缓存系统中的段持久化到硬盘，当 Translog 的数据量达到 512MB 或者 30 分钟时，会触发一次 Flush。

以减少硬盘 IO。可将 sync\_interval 的时间设置大一些。默认为5s。

如下所示。

```
PUT /{index}/_settings
{
  "index.translog.sync_interval": "5s"
}
```

"index.translog.flush\_threshold\_size" 参数的默认值是 512MB，进行如下修改：

```
PUT /{index}/_settings
{
  "index.translog.flush_threshold_size": "1024mb"
}
```

增加参数值意味着文件缓存系统中可能需要存储更多的数据，所以需要为操作系统的文件缓存系统留下足够的空间。

## 2.5. 减少副本的数量

TongSearch 为了保证集群的可用性，提供了 Replicas（副本）支持，然而每个副本也会执行分析、索引及可能的合并过程，所以 Replicas 的数量会严重影响写索引的效率。

当写索引时，需要把写入的数据都同步到副本节点，副本节点越多，写索引的效率就越慢。

如果我们需要大批量进行写入操作，可以先禁止 Replica 复制，设置 index.number\_of\_replicas: 0 关闭副本。

如下所示。

```
PUT /{index}/_settings
{
  "index.number_of_replicas": "0"
}
```

在写入完成后，Replica 修改回正常的状态。

## 2.6. 修改 index\_buffer\_size 的设置

索引缓冲的设置可以控制多少内存分配给索引进程。这是一个全局配置，会应用于一个节点上所有不同的分片上。

这是一个 static 配置项。

```
indices.memory.index_buffer_size: 10%
indices.memory.min_index_buffer_size: 48mb
```

indices.memory.index\_buffer\_size 接受一个百分比或者一个表示字节大小的值。

默认是10%，意味着分配给节点的总内存的10%用来做索引缓冲的大小。这个数值被分到不同的分片（shards）上。

如果设置的是百分比，还可以设置 min\_index\_buffer\_size（默认 48mb）和 max\_index\_buffer\_size（默认没有上限）。

### 注意:

这属于静态配置，需在每个节点上修改 tongsearch.yml 配置文件后重启生效。

## 2.7. 注意 \_id 字段的使用

\_id 字段的使用，应尽可能避免自定义 \_id，以避免针对 ID 的版本管理；建议使用 TongSearch 的默认 ID 生成策略或使用数字类型 ID 做为主键。

## 2.8. 注意 \_all 字段及 \_source 字段的使用

\_all 字段及 \_source 字段的使用，应该注意场景和需要，\_all 字段包含了所有的索引字段，方便做全文检索。若无此需求，可以禁用；\_source 存储了原始的 document 内容，如果没有获取原始文档数据的需求，可通过设置 includes、excludes 属性来定义放入 \_source 的字段。

## 2.9. 合理的配置使用 index 属性

合理的配置使用 index 属性，analyzed 和 not\_analyzed，根据业务需求来控制字段是否分词或不分词。只有 groupby 需求的字段，配置时就设置成 not\_analyzed，以提高查询或聚类的效率。

## 3. 读优化

### 3.1. 避免大结果集和深翻

要查询从 from 开始的 size 条数据，则需要每个分片中查询得分排名在前面的 from+size 条数据。

协同节点将收集到的  $n \times (from + size)$  条数据聚合，再进行一次排序，然后从 from+size 开始返回 size 条数据。

当 from、size 或者 n 中有一个值很大的时候，需要参加排序的数量也会增长，这样的查询会消耗很多 CPU 资源，从而导致效率的降低。

为了提升查询效率，TongSearch 提供了 Scroll 和 Scroll-Scan 这两种查询模式。

Scroll：是为检索大量的结果而设计的。例如，需要查询 1 ~ 100 页的数据，每页 100 条数据。

如果使用 Search 查询：每次都需要在每个分片上查询得分最高的 from+100 条数据，然后协同节点把收集到的  $n \times (from + 100)$  条数据聚合起来再进行一次排序。

每次返回 from+1 开始的 100 条数据，并且要重复执行 100 次。

如果使用 Scroll 查询：在各个分片上查询 10000 条数据，协同节点聚合  $n \times 10000$  条数据进行合并、排序，并将排名前 10000 的结果快照起来。这样做的好处是减少了查询和排序的次数。

Scroll 初始查询的命令是：

```
GET {index}/_search?scroll=2m
{
  "query": {
    "match": {
      "name": "钱丁君"
    }
  },
  "size": 20
}
```

以上查询语句的含义是，在查询字段 name 包含“钱丁君”的数据。

scroll=2m 表示下次请求的时间不能超过 2 分钟，size 表示这次和后续的每次请求一次返回的数据条数。

在这次查询的结果中除了返回了查询到的结果，还返回了一个 scroll\_id，可以把它作为下次请求的参数。

再次查询：

```
GET /_search/scroll
{
  "scroll": "2m",

  "scroll_id": "DnF1ZXJ5VGhlbkZldGNoBQAAAAAAAAAGFIB6Y3QtNk9oUmdpc09Tb21rX2NXQXcAAAAAAAAABxZ
```

```
QemN0LTZPaFJnaXNPU29ta19jV0F3AAAAAAAAAAgWUHpdC02T2hSZ2lzT1NvbWtfY1dBdwAAAAAAAAAJFIB6
Y3QtNk9oUmdpc09Tb21rX2NXQXcAAAAAAAAAChZQemN0LTZPaFJnaXNPU29ta19jV0F3"
}
```

因为这次并没有到分片里查询数据，而是直接在生成的快照里面以游标的形式获取数据。

所以这次查询并没有包含 index 和 type，也没有查询条件：

- "scroll": "2m": 指本次请求的时间不能超过 2 分钟。
- "scroll\_id": 是上次查询时返回的 scroll\_id。

## 3.2. 选择合适的路由

TongSearch 中所谓的路由和 IP 网络不同，是一个类似于 Tag 的东西。在创建文档的时候，可以通过字段为文档增加一个路由属性的 Tag。

在多分片的 TongSearch 集群中，对搜索的查询大致分为如下两种。

TongSearch 内在机制决定了拥有相同路由属性的文档，一定会被分配到同一个分片上，无论是主分片还是副本。

查询时可以根据 Routing 信息，直接定位到目标分片，避免查询所有的分片，再经过协调节点二次排序。

如果在查询条件中不包含 Routing，在查询时就遍历所有分片，整个查询主要分为 Scatter、Gather 两个过程：

- Scatter（分发）：请求到达协调节点之后，协调节点将查询请求分发给每个分片。
- Gather（聚合）：协调节点在每个分片上完成搜索，再将搜索到的结果集进行排序，将结果数据返回给用户。

使用 Routing 信息查找的效率很高，避免了多余的查询。

所以我们在设计 TongSearch Mapping 时要合理地利用 Routing 信息，来提升查询的效率。

例如，在大型的本地分类网站中，可以将城市 ID 作为 Routing 的条件，让同一个城市的数据落在相同的分片中。

默认的公式如下：

```
shard = hash(routing)%number_of_primary_shards
```

不过需要注意的是，根据城市 ID 进行分片时，也会容易出现分片不均匀的情况。

例如，大型城市的数据过多，而小城市的数据太少，导致分片之间的数据量差异很大。

这时就可以进行必要的调整，比如把多个小城市的数据合并到一个分片上，把大城市的数据按区域进行拆分到不同分配。

## 3.3. SearchType

在 Scatter、Gather 的过程中，节点间的数据传输和打分（SearchType），可以根据不同的场景选择。

如下所示。

- **QUERY\_THEN\_FETCH**: TongSearch 默认搜索方式。
  - 第一步，先向所有的分片发请求，各分片只返回文档的相似度得分和文档的 ID，然后协调节点按照各分片返回的分数进行重新排序和排名，再取出需要返回给客户端的 Size 个文档 ID。
  - 第二步，在相关的分片中取出文档的详细信息并返回给用户。
- **QUERY\_AND\_FETCH**: 协调节点向所有分片发送查询请求，各分片将文档的相似度得分和文档的详细信息一起返回。

然后，协调节点进行重新排序，再取出需要返回给客户端的数据，将其返回给客户端。由于只需要在分片中查询一次，所以性能是最好的。

- **DFS\_QUERY\_THEN\_FETCH**: 与 **QUERY\_THEN\_FETCH** 类似，但它包含一个额外的阶段：在初始查询中执行全局的词频计算，以使得更精确地打分，从而让查询结果更相关。
- **QUERY\_THEN\_FETCH** 使用的是分片内部的词频信息，而 **DFS\_QUERY\_THEN\_FETCH** 使用访问公共的词频信息，所以相比 **QUERY\_THEN\_FETCH** 性能更低。
- **DFS\_QUERY\_AND\_FETCH**: 与 **QUERY\_AND\_FETCH** 类似，不过使用的是全局的词频。

## 3.4. Filter VS Query

尽可能使用过滤器上下文 (Filter) 替代查询上下文 (Query) Query: 此文档与此查询子句的匹配程度如何？

Filter: 此文档和查询子句匹配吗？TongSearch 针对 Filter 查询只需要回答「是」或者「否」，不需要像 Query 查询一样计算相关性分数，同时 Filter 结果可以缓存。

## 3.5. 脚本 (script) 合理使用

脚本使用主要有 3 种形式，内联动态编译方式、\_script 索引库中存储和文件脚本存储的形式；一般脚本的使用场景是粗排，尽量用第二种方式先将脚本存储在 \_script 索引库中，起到提前编译，然后通过引用脚本 id，并结合 params 参数使用，即可以达到模型（逻辑）和数据进行了分离，同时又便于脚本模块的扩展与维护。

## 3.6. Cache 的设置及使用

- **QueryCache**: TongSearch 查询的时候，使用 filter 查询会使用 query cache。

若业务场景中的过滤查询比较多，建议将 querycache 设置大一些，以提高查询速度。indices.queries.cache.size: 10%（默认），可设置成百分比，也可设置成具体值，如：256mb。当然也可以禁用查询缓存（默认是开启），通过 index.queries.cache.enabled: false 设置。此配置属于 static 配置。

```
indices.queries.cache.size: 10%
```

- **FieldDataCache**: 在聚类或排序时，field data cache 会使用频繁，因此，设置字段数据缓存的大小，



在聚类或排序场景较多的情形下很有必要，可通过 `indices fielddata.cache.size: 30%` 或具体值 10GB 来设置。

但是如果场景或数据变更比较频繁，设置 cache 并不是好的做法，因为缓存加载的开销也是特别大的。此配置属于 static 配置。

```
indices.fielddata.cache.size: 30%
```

- **ShardRequestCache**: 查询请求发起后，每个分片会将结果返回给协调节点(Coordinating Node)，由协调节点将结果整合。

如果有需求，可以设置开启；通过设置 `index.requests.cache.enable: true` 来开启。不过，shard request cache只缓存hits.total,aggregations, suggestions类型的数据，并不会缓存 hits 的内容。也可以通过设置`indices.requests.cache.size: 1%`（默认）来控制缓存空间大小。

```
PUT /{index}/_settings
{ "index.requests.cache.enable": true }
```

## 3.7. 定期删除

由于在 Lucene 中段具有不变性，每次进行删除操作后不会立即从硬盘中进行实际的删除，而是产生一个 .del 文件记录删除动作。

随着删除操作的增长，.del 文件会越来越多。当进行查询操作的时候，被删除的数据还会参与检索中，然后根据 .del 文件进行过滤。.del 文件越多，查询过滤过程越长，进而影响查询的效率。

当机器空闲时，我们可以通过如下命令删除文件，来提升查询的效率：

```
$ curl -XPOST localhost:9200/chandler/_forcemerge?only_expunge_deletes=true
```

定期对不再更新的索引做 Force Merge。

这 Force Merge 的实质是对 Segment File 强制做合并，可以节省大量的 Segment Memory。

## 3.8. 更多查询优化经验

1. query\_string 或 multi\_match的查询字段越多，查询越慢。可以在mapping阶段，利用copy\_to属性将多字段的值索引到一个新字段，multi\_match时，用新的字段查询。
2. 日期字段的查询，尤其是用now 的查询实际上是不存在缓存的，因此，可以从业务的角度来考虑是否一定要用 now，毕竟利用query cache 是能够大大提高查询效率的。查询结果集的大小不能随意设置成大得离谱的值，如 query.setSize 不能设置成 Integer.MAX\_VALUE，因为 TongSearch 内部需要建立一个数据结构来放指定大小的结果集数据。
3. 避免层级过深的聚合查询，层级过深的aggregation，会导致内存、CPU 消耗，建议在服务层通过程序来组装业务，也可以通过 pipeline 的方式来优化。
4. 复用预索引数据方式来提高AGG性能：如通过 terms aggregations 替代 range aggregations，如要根据年龄来分组。



分组目标是：少年（14岁以下） 青年（14-28） 中年（29-50） 老年（51以上），可以在索引的时候设置一个 age\_group 字段，预先将数据进行分类。从而不用按 age 来做 range aggregations，通过 age\_group 字段就可以了。

## 4. 服务器配置

### 4.1. CPU 配置

一般说来，CPU 繁忙的原因有以下几个：

- 线程中有无限空循环、无阻塞、正则匹配或者单纯的计算；
- 发生了频繁的 GC；
- 多线程的上下文切换。

大多数 TongSearch 部署往往对 CPU 要求不高。因此，相对其它资源，具体配置多少个（CPU）不是那么关键。

您应该选择具有多个内核的现代处理器，常见的集群使用 2 到 8 个核的机器。

如果您要在更快的 CPU 和更多的核数之间选择，选择更多的核数更好。多个内核提供的额外并发远胜过稍微快一点点的时钟频率。

### 4.2. 堆大小的设置

TongSearch 默认安装后设置的内存是 1GB，对于任何一个现实业务来说，这个设置都太小了。

如果是通过解压安装的 TongSearch，则在 TongSearch 安装文件中包含一个 jvm.option 文件，添加如下命令来设置 TongSearch 的堆大小：

```
-Xms10g
-Xmx10g
```

Xms 表示堆的初始大小，Xmx 表示可分配的最大内存，都是 10GB。

确保 Xmx 和 Xms 的大小是相同的，其目的是为了能够在 Java 垃圾回收机制清理完堆区后不需要重新分隔计算堆区的大小而浪费资源，可以减轻伸缩堆大小带来的压力。

也可以通过命令行参数的形式，在程序启动的时候把内存大小传递给 TongSearch，如下所示。

```
./bin/tongsearch -Xmx10g -Xms10g
```

这种设置方式是一次性的，在每次启动 TongSearch 时都需要添加。

假设你有一个 64G 内存的机器，按照正常思维思考，你可能会认为把 64G 内存都给 TongSearch 比较好，但现实是这样吗，越大越好？虽然内存对 TongSearch 来说是非常重要的，但是答案是否定的！

因为 TongSearch 堆内存的分配需要满足以下两个原则：

- **不要超过物理内存的 50%：** Lucene 的设计目的是把底层 OS 里的数据缓存到内存中。

Lucene 的段是分别存储到单个文件中的，这些文件都是不会变化的，所以很利于缓存，同时操作系统也会把这些段文件缓存起来，以便更快的访问。如果设置的堆内存过大，Lucene 可用的内存将会减少，就会严重影响降低 Lucene 的全文本查询性能。

- **堆内存的大小最好不要超过 32GB：** 在 Java 中，所有对象都分配在堆上，然后有一个 Klass Pointer 指针指向它的类元数据。

这个指针在 64 位的操作系统上为 64 位，64 位的操作系统可以使用更多的内存（ $2^{64}$ ）。

在 32 位的系统上为 32 位，32 位的操作系统的最大寻址空间为 4GB（ $2^{32}$ ）。

但是 64 位的指针意味着更大的浪费，因为您的指针本身大了。浪费内存不算，更糟糕的是，更大的指针在主内存和缓存器（例如 LLC, L1 等）之间移动数据的时候，会占用更多的带宽。

Java 使用内存指针压缩（Compressed Oops）技术来解决这个问题。它的指针不再表示对象在内存中的精确位置，而是表示偏移量。

这意味着 32 位的指针可以引用 4GB 个 Byte，而不是 4GB 个 bit。也就是说，当堆内存为 32GB 的物理内存时，也可以用 32 位的指针表示。

不过，在越过那个神奇的边界 32GB 时，指针就会变为普通对象的指针，每个对象的指针都变长了，就会浪费更多的内存，降低了 CPU 的性能，还要让 GC 应对更大的内存。

事实上，当内存到达 40 ~ 40GB 时，有效的内存才相当于内存对象指针压缩技术时的 32GB 内存。

所以即便您有足够的内存，也尽量不要超过 32G，比如可以设置为 31GB：

32GB 是 TongSearch 一个内存设置限制，那如果您的机器有很大的内存怎么办呢？现在的机器内存普遍增长，甚至可以看到有 300-500GB 内存的机器。

这时我们需要根据业务场景，进行恰当内存的分配：

- **业务场景是以全文检索为主：** 依然可以给 TongSearch 分配小于 32GB 的堆内存，剩下的交给 Lucene 用作操作系统的文件系统缓存，所有的 Segment 都缓存起来，会加快全文检索。
- **业务场景中有很多的排序和聚合：** 可以考虑一台机器上创建两个或者更多 TongSearch 节点，而不要部署一个使用 32+GB 内存的节点。

仍然要坚持 50% 原则，假设您有个机器有 128G 内存，可以创建两个 Node，使用 32G 内存。也就是说 64G 内存给 TongSearch 的堆内存，剩下的 64G 给 Lucene。

## 4.3. 服务器配置的选择

**Swapping 是性能的坟墓：** 在选择 TongSearch 服务器时，要尽可能地选择与当前应用场景相匹配的服务器。

如果服务器配置很低，则意味着需要更多的节点，节点数量的增加会导致集群管理的成本大幅度提高。

如果服务器配置很高，而在单机上运行多个节点时，也会增加逻辑的复杂度。

在计算机中运行的程序均需在内存执行，若内存消耗殆尽将导致程序无法进行。为了解决这个问题，操作系统使用一种叫作虚拟内存的技术。

当内存耗尽时，操作系统就会自动把内存中暂时不使用的数据交换到硬盘中，需要使用的时候再从硬盘交换到内存。

如果内存交换到磁盘上需要 10 毫秒，从磁盘交换到内存需要 20 毫秒，那么多的操作时延累加起来，将导

致几何级增长。

不难看出 Swapping 对于性能是多么可怕。所以为了使 TongSearch 有更好等性能，强烈建议关闭 Swap。关闭 Swap 的方式可以参考《TongSearch\_V3.0产品安装手册》。

## 4.4. GC 设置

老的 JDK 版本推荐默认设置为：Concurrent-Mark and Sweep（CMS）。

您使用的 JDK8 较高版本，或者JDK9+，推荐您使用G1 GC；因为目前的项目使用的就是 G1 GC，运行效果良好，对 Heap 大对象优化尤为明显。修改 jvm.options 文件，将下面几行。

在 jvm.options 文件中修改配置：

```
-XX:+UseG1GC
-XX:MaxGCPauseMillis=50
```

其中，-XX:MaxGCPauseMillis 是控制预期的最高GC时长，默认值为 200ms。若线上业务特性对于GC停顿非常敏感，可以适当设置低一些。但是这个值如果设置过小，可能会带来比较高的 CPU 消耗。

G1 对于集群正常运作的情况下减轻 GC 停顿对服务时延的影响还是很有效的，但是如果是你描述的GC导致集群卡死，那么很有可能换G1 也无法根本上解决问题。通常都是集群的数据模型或者 Query 需要优化。

## 4.5. 磁盘选择

硬盘对所有的集群都很重要，对大量写入的集群更是加倍重要（例如那些存储日志数据的）。硬盘是服务器上最慢的子系统，这意味着那些写入量很大的集群很容易让硬盘饱和，使得它成为集群的瓶颈。

尽量使用固态硬盘（SSD）。固态硬盘相比于任何旋转介质（机械硬盘，磁带等），无论随机写还是顺序写，都会对 IO 有较大的提升。



- 如果您正在使用 SSD，确保你的系统 I/O 调度程序是配置正确的。当你向硬盘写数据，I/O 调度程序决定何时把数据实际发送到硬盘。大多数默认 \*nix 发行版下的调度程序都叫做 cfq（完全公平队列）。
- 调度程序分配时间片到每个进程。并且优化这些到硬盘的众多队列的传递。但它是为旋转介质优化的：机械硬盘的固有特性意味着它写入数据到基于物理布局的硬盘会更高效。
- 这对 SSD 来说是低效的，尽管这里没有涉及到机械硬盘。但是，deadline 或者 noop 应该被使用。

deadline 调度程序基于写入等待时间进行优化，noop 只是一个简单的 FIFO 队列。

这个简单的更改可以带来显著的影响。仅仅是使用正确的调度程序，我们看到了 500 倍的写入能力提升。

如果您使用旋转介质（如机械硬盘），尝试获取尽可能快的硬盘（高性能服务器硬盘，15k RPM 驱动器）。

使用 RAID0 是提高硬盘速度的有效途径，对机械硬盘和 SSD 来说都是如此。

没有必要使用镜像或其它 RAID 变体，因为 TongSearch 在自身层面通过副本，已经提供了备份的功能，所以不需要利用磁盘的备份功能，同时如果使用磁盘备份功能的话，对写入速度有较大的影响。

最后，避免使用网络附加存储（NAS）。人们常声称他们的 NAS 解决方案比本地驱动器更快更可靠。除却

这些声称，从没看到 NAS 能配得上它的大肆宣传。NAS 常常很慢，显露出更大的延时和更宽的平均延时方差，而且它是单点故障的。

## 4.6. 角色隔离

### 角色隔离

TongSearch 集群中的数据节点负责对数据进行增、删、改、查和聚合等操作，所以对 CPU、内存和 I/O 的消耗很大。

在搭建 TongSearch 集群时，应该对 TongSearch 集群中的节点进行角色划分和隔离。

候选主节点，修改 tongsearch.yml 配置文件：

```
node.master=true
node.data=false
```

数据节点，修改 tongsearch.yml 配置文件：

```
node.master=false
node.data=true
```

## 4.7. 通过开启慢查询配置定位慢查询

不论是数据库还是搜索引擎，对于问题的排查，开启慢查询日志是十分必要的，TongSearch 开启慢查询的方式有多种，常调用 API 进行全局设置：

```
PUT /_all/_settings
{
  "index.search.slowlog.threshold.query.warn":"5s",
  "index.search.slowlog.threshold.query.info":"2s",
  "index.search.slowlog.threshold.query.debug":"1s",
  "index.search.slowlog.threshold.query.trace":"400ms",
  "index.search.slowlog.threshold.fetch.warn":"1s",
  "index.search.slowlog.threshold.fetch.info":"800ms",
  "index.search.slowlog.threshold.fetch.debug":"500ms",
  "index.search.slowlog.threshold.fetch.trace":"200ms",
  "index.indexing.slowlog.threshold.index.warn":"5s",
  "index.indexing.slowlog.threshold.index.info":"2s",
  "index.indexing.slowlog.threshold.index.debug":"1s",
  "index.indexing.slowlog.threshold.index.trace":"400ms"
}
```

这样，在日志目录下的慢查询日志就会有输出记录必要的信息了。

```
{CLUSTER_NAME}_index_indexing_slowlog.log
{CLUSTER_NAME}_index_search_slowlog.log
```

## 4.7.1. 慢查询日志字段说明

| 属性                    | 说明                                                                         |
|-----------------------|----------------------------------------------------------------------------|
| timestamp             | 事件发生的时间（采用 ISO9601 格式）。                                                    |
| node.name             | 节点的名称。这可以在配置 tongsearch.yml 文件中更改。                                         |
| node.id               | 节点 ID。这是自动生成的，并且在整个集群重新启动后保持不变。                                            |
| id                    | 如果使用X-Opaque-Id头发起请求，则会记录在id中                                              |
| agent                 | sdk客户端类型以及版本，当使用Tongsearch Client或者Elasticsearch Client 调用时候，该值记录所使用的类型及版本 |
| took                  | 响应耗时                                                                       |
| total_hits            | 响应总条数                                                                      |
| total_hits_size_bytes | 响应总数据大小（单位：字节）                                                             |
| search_type           | 查询类型，query_then_fetch或者dfs_query_then_fetch                                |
| source                | 原始查询语句                                                                     |

## 4.7.2. 慢查询日志转存

慢查询日志支持将日志数据转发存入当前集群固定的索引别名tongsearch\_console\_slowlog中，要使用该功能请先确保已经安装了 log-dispatch 插件。要开启转发，请确保别名tongsearch\_console\_slowlog已经创建并关联了索引和ilm。

### 4.7.2.1. 创建存储慢查询日志的索引

#### 1. 创建 ILM 策略（slowlogdata\_ilm\_policy）

- 热阶段（hot phase） - 索引刚创建就处于这个阶段，会在30天后进行滚动更新。
- 删除阶段（delete phase） - 索引满 60 天后，会被自动删除。

```
PUT /_ilm/policy/slowlogdata_ilm_policy
{
  "policy": {
    "phases": {
      "hot": {
        "min_age": "0ms",
        "actions": {
          "rollover": {
            "max_age": "30d"
          }
        }
      }
    }
  }
}
```

```

    }
  }
},
"delete": {
  "min_age": "60d",
  "actions": {
    "delete": {}
  }
}
}
}
}
}
}
}

```

## 2. 创建索引模板 (slowlogdata\_template)

此模板会应用到所有以 .tongsearch\_slowlog- 开头的索引上。

把前面定义的 ILM 策略和滚动更新别名 tongsearch\_console\_slowlog 关联起来。

```

PUT /_index_template/slowlogdata_template
{
  "index_patterns": [
    ".tongsearch_slowlog-*"
  ],
  "template": {
    "settings": {
      "index.lifecycle.name": "slowlogdata_ilm_policy",
      "index.lifecycle.rollover_alias": "tongsearch_console_slowlog"
    }
  }
}

```

## 3. 创建一个初始索引并指向别名 tongsearch\_console\_slowlog

```

PUT /.tongsearch_slowlog-000001
{
  "aliases": {
    "tongsearch_console_slowlog": {
      "is_write_index": true
    }
  }
}

```

### 4.7.2.2. 配置慢查询转发

设置slowlog.dispatch.enabled: true开启慢查询转发：

```
PUT /_slowlog/dispatch/config
{
  "slowlog.dispatch.max_retry_times": 3,
  "slowlog.dispatch.enabled": true,
  "slowlog.dispatch.rate_limit": 100,
  "slowlog.dispatch.max_queue_size": 2000
}
```

- slowlog.dispatch.max\_retry\_times: 存储失败重试次数，默认3次；
- slowlog.dispatch.enabled: 是否开启存储，默认 false；
- slowlog.dispatch.rate\_limit: 每秒钟转发数，默认每秒钟可以转发 100 条日志；
- slowlog.dispatch.max\_queue\_size: 转发缓冲队列最大长度，默认 2000；

通过以上配置，当开启慢查询日志后，慢查询日志会同时转发存储到索引 tongsearch\_console\_slowlog 里。

### 4.7.2.3. 存储的慢查询日志字段说明

| 属性             | 说明                                                                         |
|----------------|----------------------------------------------------------------------------|
| timestamp      | 事件发生的时间（采用 ISO9601 格式）。                                                    |
| node.name      | 节点的名称。这可以在配置 tongsearch.yml 文件中更改。                                         |
| node.id        | 节点 ID。这是自动生成的，并且在整个集群重新启动后保持不变。                                            |
| cluster.name   | 集群名称                                                                       |
| host.ip        | 节点所在主机ip。                                                                  |
| host.name      | 节点所在主机的主机名。                                                                |
| origin.address | 客户端网络地址。                                                                   |
| trace.id       | 如果使用X-Opaque-Id头发起请求，则会记录在trace.id中                                        |
| agent          | sdk客户端类型以及版本，当使用Tongsearch Client或者Elasticsearch Client 调用时候，该值记录所使用的类型及版本 |
| index_name     | 索引名                                                                        |
| index_uuid     | 索引uuid                                                                     |
| index_shard_id | 索引分片id                                                                     |
| total_shards   | 总分片数                                                                       |
| took           | 响应耗时                                                                       |



| 属性                     | 说明                                                                                        |
|------------------------|-------------------------------------------------------------------------------------------|
| took_millis            | 执行毫秒数                                                                                     |
| total_hits             | 响应总条数                                                                                     |
| total_hits_size_bytes  | 响应总数据大小（单位：字节）                                                                            |
| search_type            | 查询类型，query_then_fetch或者dfs_query_then_fetch                                               |
| search_scroll_id       | 查询时候指定的scroll id                                                                          |
| search_profile         | 否启用了profile                                                                               |
| types                  | 查询时候指定的_type，如使用/documents/_doc/_search进行查询，types将是_doc                                   |
| search_timeout         | 查询时候指定的超时时间                                                                               |
| source                 | 原始查询语句                                                                                    |
| stats                  | 请求中指定的stats参数                                                                             |
| index_store_size_bytes | 索引存储空间，单位字节                                                                               |
| type                   | 慢日志类型，index_search_slowlog表示是索引查询慢日志                                                      |
| message                | 由索引名和分片id组成                                                                               |
| component              | 慢日志来源，index.search.slowlog.query表示来自慢查询日志的query，index.search.slowlog.fetch表示来自慢查询日志的fetch |

#### 4.7.2.4. 存储的索引数据时产生的慢日志字段说明

| 属性           | 说明                                 |
|--------------|------------------------------------|
| timestamp    | 事件发生的时间（采用 ISO9601 格式）。            |
| node.name    | 节点的名称。这可以在配置 tongsearch.yml 文件中更改。 |
| node.id      | 节点 ID。这是自动生成的，并且在整个集群重新启动后保持不变。    |
| cluster.name | 集群名称。                              |
| cluster.uuid | 集群 uuid。                           |
| host.ip      | 节点所在主机 ip。                         |

| 属性             | 说明                                                                          |
|----------------|-----------------------------------------------------------------------------|
| host.name      | 节点所在主机的主机名。                                                                 |
| origin.address | 客户端网络地址。                                                                    |
| trace.id       | 如果使用 X-Opaque-Id 头发起请求，则会记录在 trace.id 中。                                    |
| agent          | sdk客户端类型以及版本，当使用Tongsearch Client或者Elasticsearch Client 调用时候，该值记录所使用的类型及版本。 |
| index_name     | 索引名。                                                                        |
| index_uuid     | 索引 uuid。                                                                    |
| took           | 响应耗时。                                                                       |
| took_millis    | 执行毫秒数。                                                                      |
| source         | 文档 source。                                                                  |
| doc_type       | 文档类型。                                                                       |
| type           | 慢日志类型，index_indexing_slowlog 表示是索引数据慢日志。                                    |
| doc_id         | 文档 id。                                                                      |
| doc_routing    | 指定的文档路由值。                                                                   |
| message        | 由索引名称和索引 uuid 组成。                                                           |
| component      | 慢日志来源，index_indexing_slowlog.index 表示来自索引数据慢日志。                             |

## 5. 数据结构优化

基于 TongSearch 的使用场景，文档数据结构尽量和使用场景进行结合，去掉没用及不合理的数据。

### 5.1. 尽量减少不需要的字段

如果 TongSearch 用于业务搜索服务，一些不需要用于搜索的字段最好不存到 TS 中，这样即节省空间，同时在相同的数据量下，也能提高搜索性能。

避免使用动态值作字段，动态递增的 mapping，会导致集群崩溃；同样，也需要控制字段的数量，业务中不使用的字段，就不要索引。控制索引的字段数量、mapping 深度、索引字段的类型，对于 TS 的性能优化是重中之重。以下是 TS 关于字段数、mapping 深度的一些默认设置：

```

PUT /{index}/_settings
{
  "index.mapping.nested_objects.limit": "10000",
  "index.mapping.total_fields.limit": "1000",
  "index.mapping.depth.limit": 20
}
```

### 5.2. 嵌套对象与父/子对象

尽量避免使用 nested 或 parent/child 的字段，能不用就不用；nested query 慢，parent/child query 更慢，比 nested query 慢上百倍；因此能在 mapping 设计阶段搞定的（大宽表设计或采用比较 smart 的数据结构），就不要用父子关系的 mapping。

如果一定要使用 nested fields，保证 nested fields 字段不能过多，目前 TS 默认限制是 50。因为针对 1 个 document，每一个 nested field，都会生成一个独立的 document，这将使 doc 数量剧增，影响查询效率，尤其是 JOIN 的效率。

```

PUT /{index}/_settings
{
  "index.mapping.nested_fields.limit": "50"
}
```

| 对比 | 嵌套对象               | 父/子                          |
|----|--------------------|------------------------------|
| 优点 | 文档存储在一起，因此读取性高     | 父子文档可以独立更新，互不影响              |
| 缺点 | 更新父文档或子文档时需要更新整个文档 | 为了维护 join 关系，需要占用部分内存，读取性能较差 |
| 场景 | 子文档偶尔更新，查询频繁       | 子文档更新频繁                      |

## 5.3. 选择静态映射、非必需时，禁止动态映射

尽量避免使用动态映射，这样有可能会造成集群崩溃，此外，动态映射有可能会带来不可控制的数据类型，进而有可能导致在查询端出现相关异常，影响业务。

此外，TongSearch 作为搜索引擎时，主要承载 query 的匹配和排序的功能，那数据的存储类型基于这两种功能的用途分为两类，一是需要匹配的字段，用来建立倒排索引对 query 匹配用，另一类字段是用做粗排用到的特征字段，如 ctr、点击数、评论数等等。

## 5.4. document 模型设计

对于 MySQL，我们经常有一些复杂的关联查询。在 TongSearch 里该怎么处理，TongSearch 里面的复杂的关联查询尽量别用，一旦用了性能一般都不太好。最好是先在 Java 系统里就完成关联，将关联好的数据直接写入 TongSearch 中。

搜索的时候，就不需要利用 TongSearch 的搜索语法来完成 join 之类的关联搜索了。

document 模型设计是非常重要的，很多操作，不要在搜索的时候才想去执行各种复杂的乱七八糟的操作。

TongSearch 能支持的操作就那么多，不要考虑用 TongSearch 做一些它不好操作的事情。若真的存在那种操作，尽量在 document 模型设计的时候，写入的时候就完成。另外对于一些太复杂的操作，比如 join/nested/parent-child 搜索都要尽量避免，性能都很差。

## 6. 集群架构设计

合理的部署 TongSearch 有助于提高服务的整体可用性。

### 6.1. 主节点、数据节点和协调节点分离

TongSearch 集群在架构拓朴时，采用主节点、数据节点和负载均衡节点分离的架构，又可将数据节点再细分为“Hot-Warm”的架构模式。

TongSearch 的配置文件中有 2 个参数，node.master 和 node.data。这两个参数搭配使用时，能够帮助提供服务器性能。

#### 主 (master) 节点

配置 node.master:true 和 node.data:false，该 node 服务器只作为一个主节点，但不存储任何索引数据。

推荐每个集群运行 3 个专用的 master 节点来提供最好的弹性。用 3 个专用的 master 节点，专门负责处理集群的管理以及加强状态的整体稳定性。

因为这 3 个 master 节点不包含数据也不会实际参与搜索以及索引操作，在 JVM 上它们不用做相同的事，例如繁重的索引或者耗时，资源耗费很大的搜索。因此不太可能会因为垃圾回收而导致停顿。因此，master 节点的 CPU，内存以及磁盘配置可以比 data 节点少很多的。

#### 数据 (data) 节点

配置 node.master:false 和 node.data:true，该 node 服务器只作为一个数据节点，只用于存储索引数据，使该 node 服务器功能单一，只用于数据存储和数据查询，降低其资源消耗率。

data 节点又可再细分为“Hot-Warm”架构，即分为热节点 (hot node) 和暖节点 (warm node)。

- hot 节点

hot 节点主要是索引节点 (写节点)，同时会保存近期的一些频繁被查询的索引。由于进行索引非常耗费 CPU 和 IO，即属于 IO 和 CPU 密集型操作，建议使用 SSD 的磁盘类型，保持良好的写性能；我们推荐部署最小化的 3 个 hot 节点来保证高可用性。

根据近期需要收集以及查询的数据量，可以增加服务器数量来获得想要的性能。

将节点设置为 hot 类型需要 tongsearch.yml 如下配置：

```
node.attr.box_type: hot
```

如果是针对指定的 index 操作，可以通过 settings 设置 index.routing.allocation.require.box\_type: hot 将索引写入 hot 节点。

```
PUT /{index}/_settings
{
  "index.routing.allocation.require.box_type": "hot"
}
```

- warm 节点

这种类型的节点是为了处理大量的，而且不经常访问的只读索引而设计的。由于这些索引是只读的，warm 节点倾向于挂载大量磁盘（普通磁盘）来替代 SSD。内存、CPU 的配置跟 hot 节点保持一致即可；节点数量一般也是大于等于 3 个。

将节点设置为 warm 类型需要 tongsearch.yml 如下配置：

```
node.attr.box_type: warm
```

同时，也可以在 tongsearch.yml 中设置 index.codec:best\_compression 保证 warm 节点的压缩配置。

当索引不再被频繁查询时，可通过 index.routing.allocation.require.box\_type:warm，将索引标记为 warm，从而保证索引不写入 hot 节点，以便将 SSD 磁盘资源用在刀刃上。一旦设置这个属性，TongSearch 会自动将索引合并到 warm 节点。

- 协调（coordinating）节点

协调节点用于做分布式里的协调，将各分片或节点返回的数据整合后返回。该节点不会被选作主节点，也不会存储任何索引数据。该服务器主要用于查询负载均衡。在查询时，通常会涉及到从多个 node 服务器上查询数据，并将请求分发到多个指定的 node 服务器，并对各个 node 服务器返回的结果进行一个汇总处理，最终返回给客户端。

在 TongSearch 集群中，所有的节点都有可能是协调节点，但是，可以通过设置 node.master、node.data、node.ingest 都为 false 来设置专门的协调节点。需要较好的 CPU 和较高的内存。

- node.master:false 和 node.data:true，该 node 服务器只作为一个数据节点，只用于存储索引数据，使该 node 服务器功能单一，只用于数据存储和数据查询，降低其资源消耗率。
- node.master:true 和 node.data:false，该 node 服务器只作为一个主节点，但不存储任何索引数据，该 node 服务器将使用自身空闲的资源，来协调各种创建索引请求或者查询请求，并将这些请求合理分发到相关的 node 服务器上。
- node.master:false 和 node.data:false，该 node 服务器即不会被选作主节点，也不会存储任何索引数据。该服务器主要用于查询负载均衡。

在查询时，通常会涉及到从多个 node 服务器上查询数据，并将请求分发到多个指定的 node 服务器，并对各个 node 服务器返回的结果进行一个汇总处理，最终返回给客户端。

## 6.2. 关闭 data 节点服务器中的 http 功能

针对 TongSearch 集群中的所有数据节点，不用开启 http 服务。将其中的配置参数这样设置“http.enabled:false”，同时也不要安装 head、bigdesk、marvel 等监控插件，这样保证 data 节点服务器只需处理创建/更新/删除/查询索引数据等操作。

http 功能可以在非数据节点服务器上开启，上述相关的监控插件也安装到这些服务器上，用于监控 TongSearch 集群状态等数据信息。

这样做一来出于数据安全考虑，二来出于服务性能考虑。

## 6.3. 一台服务器上最好只部署一个 node

一台物理服务器上可以启动多个 node 服务器节点（通过设置不同的启动 port），但一台服务器上的 CPU、内存、硬盘等资源毕竟有限，从服务器性能考虑，不建议一台服务器上启动多个 node 节点。

## 6.4. 集群分片设置

TS 一旦创建好索引后，就无法调整分片的设置，而在 TS 中，一个分片实际上对应一个 lucene 索引，而 lucene 索引的读写会占用很多的系统资源，因此，分片数不能设置过大；所以，在创建索引时，合理配置分片数是非常重要的。一般来说，我们遵循一些原则：

控制每个分片占用的硬盘容量不超过 TS 的最大 JVM 的堆空间设置（一般设置不超过 32G），因此，如果索引的总容量在 500G 左右，那分片大小在 16 个左右即可；考虑一下 node 数量，一般一个节点有时候就是一台物理机。

如果分片数过多，大大超过了节点数，很可能会导致一个节点上存在多个分片，一旦该节点故障，即使保持了 1 个以上的副本，同样有可能导致数据丢失，集群无法恢复。所以，一般都设置分片数不超过节点数的 3 倍。

